

ConsultBae AI Automation Assignment

Task 4: Data Issues Report

Prepared by Sanchay Roy

Executive Summary

The three source files contain 103 rows in total (42 in source1_naukri_applicants.csv, 31 in source2_gig_workers.csv, 30 in source3_cbnexus_contacts.csv). After running the merge pipeline, these 103 rows resolve to 53 canonical people. 42 rows matched an existing person with high confidence, 53 rows became new people, and 8 rows were flagged for manual review instead of being merged automatically, because merging them would have meant guessing. Of the 53 people, 25 are represented in two or more source files, and 15 are represented in all three.

Below is every data quality problem I actually found in the three files, with the real example that shows it, and exactly what my code does about it: fixed, normalized, merged, flagged, or left alone.

1. Overview of the Three Sources

source1_naukri_applicants.csv has Full Name, Email, Phone, City, Experience, Current CTC, Applied Date, and Skills. It is the only file with both an email and a phone number, so I used it as the anchor table.

source2_gig_workers.csv has email_id, worker_name, rate, location, status, and skill_tags. It has an email but no phone number.

source3_cbnexus_contacts.csv has Name, Phone Number, City, Verified, and Projects Completed. It has a phone number but no email.

Because no single column is common to all three files, I could not just do one join. Email bridges source1 and source2, and phone bridges source1 and source3. People who only appear in source2 and source3, with no row in source1, have no shared identifier field between them at all (one side only has email, the other only has phone), which turned out to be the hardest part of the matching, covered in section 6.

2. Matching Strategy (what the code actually does)

Every raw row is normalized before comparison: names are lowercased with punctuation stripped, emails are lowercased and trimmed, phone numbers are reduced to the last bare digits after stripping a +91 or 91 country prefix or a leading 0, so all three phone formats in the data collapse to the same value.

For each row, I check it against every canonical person seen so far:

- If the normalized email matches a person's known email, or the normalized phone matches a person's known phone, that is a strong identifier match.
- Name compatibility is checked separately: two names are compatible if they are identical after normalization, or if they share the same last name and the first names match or one is an initial of the other (for example 'r verma' and 'rohit verma'), or if one name is a substring of the other.

I want to be clear about what this is not: it is not fuzzy string matching or edit-distance matching. It is a fixed set of compatibility rules. I chose this over something like Levenshtein distance because I wanted matching decisions I could fully explain and defend, not a similarity score I would have to guess a threshold for.

Based on these checks, every row gets one of three decisions:

- **MATCH** (high confidence): the email or phone matches an existing person exactly, and the name is compatible with no conflict. The row is merged into that person.
- **NEW_PERSON**: no existing person matches on email, phone, or name. A new canonical person is created.
- **REVIEW** (low confidence): something does not line up cleanly, for example the name matches a person but the email or phone conflicts with what that person already has, or the name matches but there is no shared email or

phone at all to confirm it. These rows are not merged and not turned into a new person. They are kept in the database as unresolved, linked to nothing, and logged separately so a human can look at them.

This last point matters: right now, the 8 REVIEW rows are not represented as their own people either. Their raw data is preserved, but they are not yet part of the finished people table. See section 8 for why I chose this over guessing, and section 11 for the limitation this creates.

3. Source 1 Issues (source1_naukri_applicants.csv)

Issue	File	Example from the data	What my code does about it
Phone numbers in four different formats	source1	Row 10: +919000000222 Row 11: 09000000167 Row 17: 9000000106	All are stripped to digits only, then a leading 91 or a leading 0 is removed if it leaves a 10-digit number. All four formats normalize to the same kind of value, which is what makes the source1-source3 phone match work at all.
City names inconsistent in case and spelling, and with stray whitespace	source1	Column has: NOIDA, Noida, 'Noida ' (trailing space), GURGAON, Gurugram, bangalore, Bengaluru, 'new delhi', 'Delhi NCR'	City is lowercased and trimmed for storage. Bangalore/Bengaluru and Gurgaon/Gurugram are mapped to one spelling. I did not merge Delhi, New Delhi, and Delhi NCR into one value, they stay as three separate normalized strings. This does not affect matching correctness because city is never used to decide whether two rows are the same person, it is stored for display only. It does mean the city field itself is not fully clean, which I am flagging rather than hiding.
Current CTC mixes two different units with no unit flag	source1	Row 2: 417964 (looks like a raw annual rupee figure) Row 6: 4.2 (looks like lakhs per year)	I used a threshold: any value under 100 is treated as LPA and multiplied by 100000, anything 100 or above is treated as already being an absolute rupee figure. This is a heuristic based on what a plausible salary looks like in each unit, not a certainty. It works for every row in this dataset, but it would break on a genuinely tiny raw salary or a CTC above 100 lakh, neither of which occurs here.
Applied Date uses at least four different date formats in the same column	source1	Row 2: 24-07-2026 (DD-MM-YYYY) Row 3: 2026-08-08 (YYYY-MM-DD) Row 5: 7 Jul 2026 Row 7: 07/13/2026 (day over 12, so unambiguously MM/DD/YYYY)	The parser tries a list of known formats in order and converts to YYYY-MM-DD. If a value does not match any of them it is kept as the raw string instead of being dropped, so no date silently disappears, but an unparseable date would not be sorted correctly either. No such row actually occurred in this file.
Same person entered twice with a shortened name, under the exact same email and phone	source1	Row 25: 'R. Verma' Row 31: 'Rohit Verma', both rohit.verma13@mailtest.example.org , both 9000000294	Because the email is identical, this is a high-confidence match and both rows are merged into one canonical person automatically. My audit script also lists it separately as a suspicious duplicate within the same file, since two rows for the same email in one source is worth a second look.
A second 'Nikhil Chopra' entered under a different email but the same phone number	source1	Row 27: alt.nikhil.chopra70@example.com, phone 09000000103 Row 37: nikhil.chopra70@example.com, phone 09000000103	This is not automatically merged. The phone matches, but the email is different from the one already on file for that person, so it is flagged as an identifier conflict and left for review instead of being merged or silently treated as a different person. I would rather

			have one extra row to check by hand than risk two different people getting fused by a false phone match, or one real person getting split by a false rejection.
--	--	--	---

4. Source 2 Issues (source2_gig_workers.csv)

Issue	File	Example from the data	What my code does about it
One completely blank row	source2	Line 12 is empty (just commas, no values)	Detected and skipped during loading, it never reaches the matching stage.
One row has its columns shifted left by one	source2	Line 20: "react, javascript, mysql",ISHA.CHOPRA95@MAILTEST.EXAMPLE.ORG,Isha Chopra,1406/hr,Pune,active', the skill list sits where the email should be	Detected by checking whether the first field looks like an email. When it does not, the six fields are re-read shifted by one position (skills moved to the front, email moved to the second slot, and so on) and reconstructed correctly. Once realigned, this row's email matches the Isha Chopra entered on line 7 of the same file, so it merges as a duplicate of an existing person rather than being dropped. My audit script also flags this pairing as a suspicious duplicate for a human to double check.
Email casing is inconsistent	source2	Line 7 and line 15: ISHA.CHOPRA95@MAILTEST.EXAMPLE.ORG and DEEPAK.NAIR44@EXAMPLE.COM in uppercase, most others lowercase	Every email is lowercased before comparison, so casing differences do not break the email match against source1.
Rate mixes hourly and monthly figures with no shared unit	source2	Line 2: 1415/hr Line 10: 72k/month	The value is parsed into a numeric amount and a unit ('hr' or 'month'), with a 'k' suffix expanded to thousands. I did not convert everything into one common unit, because turning an hourly rate into a monthly figure needs an assumed number of working hours per month, and I had no real basis for picking one, so I kept the two as separate fields instead of inventing a conversion.
Status has inconsistent casing, and a third value that is not simply active or inactive	source2	Active / active / ACTIVE all appear, and separately: 'paused' (lines 11, 15, 26)	Status is lowercased for consistency. I kept 'paused' as its own distinct value rather than folding it into inactive, since a paused gig worker is not the same thing as an inactive one.

5. Source 3 Issues (source3_cbnexus_contacts.csv)

Issue	File	Example from the data	What my code does about it
The header row appears a second time in the middle of the file, as if it were a data row	source3	Line 16 repeats the exact header: 'Name,Phone Number,City,Verified,Projects Completed'	Detected by comparing each row against the header row and skipping an exact match. This is the same check for every load, so it would also catch the header being repeated anywhere else in the file, not just line 16.
Phone numbers appear in three different formats	source3	Line 2: 9000000268 (plain) Line 4: 919000000231 (91 prefix, no plus) Line 5: +91-9000000131 (plus and hyphen)	Same normalization as source1: strip everything but digits, then remove a leading 91 or leading 0 if it leaves 10 digits. This is what lets source3 rows match source1 rows by phone even though the raw formats look nothing alike.

Verified is written as four different literal values for what is really a yes/no field	source3	Y, N, yes, No all appear (rows 2, 3, 4, 9)	Normalized to a plain true/false. Y and yes both become true, N and no both become false.
Some names are in all caps	source3	Rows 3, 7, 9, 14, 19: RITU SHARMA, RAHUL MALHOTRA, SAHIL MALHOTRA, KARAN BHATIA, MEERA BHATIA, mixed with normal-case names elsewhere	The raw name is kept as-is for display, but the normalized version used for matching is lowercased, so casing never affects whether two names are treated as compatible.

6. Cross-Source Matching Issues: People Who Share a Name

The dataset has a real trap in it: several names are not unique. 'Arjun Mehta' appears three separate times across the files with three different phone numbers, and 'Deepak Nair' appears twice with two different email addresses. These are different people who happen to share a name, not duplicates of the same person, and matching on name alone would have wrongly fused them.

Arjun Mehta: source1 row 20 (arjun.mehta9@example.in, phone 9000000131) and source3 row 5 (phone +91-9000000131) share the same phone once normalized, so those two rows correctly merge into one person. Source2 row 18 has a different email, arjun.mehta77@mailtest.example.org, and no phone at all. When it is checked against the already-merged person, the name matches but the email conflicts with the email already on file, so it is flagged for review, not merged. Separately, source3 row 28 has a third phone number, 9000000272, again a name match but a conflicting phone, again flagged for review rather than merged or guessed.

Deepak Nair follows the same pattern: source1, source2 (line 15), and source3 agree on deepak.nair44@example.com and one phone number and merge cleanly. A second source2 row (line 32) uses a different email, DEEPAK.NAIR57@EXAMPLE.IN, and is flagged for review instead of being merged in.

In both cases, the system did the conservative, defensible thing: it refused to guess which Arjun Mehta or which Deepak Nair a name-only or conflicting-identifier row belonged to, and left it for a human to resolve.

7. Cross-Source Matching Issues: No Shared Identifier At All

A second, harder case: some people appear only in source2 and source3, and nowhere in source1. Since source2 only has email and source3 only has phone, there is no field at all in common between their two rows for that person, only the name.

This affects at least four people in the data: Manish Bhatia, Divya Chopra, Karan Chopra, and Vikram Mehta. For each of them, the source3 row's name matches the person already created from their source2 row, but there is no email or phone overlap to confirm it, so each is flagged as a name-only review case rather than merged. City was not used to help resolve these, because city is not part of the matching logic at all in this implementation, it is stored for display only.

8. Duplicate Handling Summary

Two duplicates were detected and merged automatically because they shared a strong identifier: the R. Verma / Rohit Verma pair in source1 (same email), and the shifted Isha Chopra row in source2 once its columns were realigned (same email as an earlier row in the same file).

Eight rows were flagged for review rather than merged: four because a name matched an existing person but the email or phone conflicted (Nikhil Chopra, the two extra Arjun Mehta rows, the extra Deepak Nair row), and four because a name matched but there was no shared identifier at all to confirm it (Manish Bhatia, Divya Chopra, Karan Chopra, Vikram Mehta). None of these eight were merged, and none of them were silently treated as confident new people either. They sit in the database unresolved, with their raw data preserved, so nothing is lost, but nothing is guessed.

9. Missing Data Handling

The one fully blank row in source2 (line 12) is skipped during loading. Individual missing fields, such as a blank skills or experience value on an otherwise valid row, are kept as empty rather than causing the row to be dropped, since a missing skill list does not mean the person themselves is invalid data.

10. Malformed Data Handling

Two structural problems were detected and corrected rather than dropped: the shifted-column row in source2 (line 20) was realigned field by field based on which position actually looked like an email, and the duplicate header row in source3 (line 16) was detected by comparing the row's values to the header itself and skipped.

11. Remaining Limitations

- The 8 REVIEW rows are not currently represented as canonical people at all. Their data is preserved in the raw records table and logged for review, but if, for example, arjun.mehta77@mailtest.example.org really is a different person from the Arjun Mehta already in the system, that person does not yet have their own entry in the finished people table. Resolving that would need either a human decision or a stronger signal than the data currently gives (a shared field, or independently verified identity).
- The CTC LPA-versus-raw-rupee threshold at 100 is a reasonable heuristic for this dataset, not a guaranteed rule. It has not been individually verified against every row.
- Gig rate is kept as separate amount and unit fields (hourly or monthly) rather than converted to one common unit, since I had no reliable basis for an hours-per-month assumption.
- City normalization only merges two known alias pairs (Bangalore/Bengaluru and Gurgaon/Gurugram). Delhi, New Delhi, and Delhi NCR are left as three separate values. This does not affect matching, since city is not used as a matching signal in this implementation, but it does mean the city field itself is not fully normalized.
- Name compatibility uses fixed rules (exact match, shared last name with matching or initial first name, or substring containment), not a fuzzy or edit-distance match. A name with a typo that does not fit one of those patterns would not be recognized as compatible.

12. Conclusion

Of 103 raw rows, 95 were confidently resolved, either merged into an existing person or created as a new one, and 8 were deliberately left unresolved because merging them would have meant guessing. I would rather hand over 8 flagged rows with a clear reason each than silently merge two different Arjun Mehtas or two different Deepak Nairs into one record.